

Chapter 2

Sound File Format

Ext	Type	Name	Content
94L	ASCII	list	list of 94I with associated MIDI prg chg & variation
94I	RIFF BIN	instrument	instrument parameter
94K	RIFF merge	bank	merged 94L 94I & samples
94B	RIFF bin	binary	merged parameters, samples, reloc table
SMP	BIN	sample	Sample Vision format samples
WAV	RIFF	wave	Microsoft format samples
RMI	RIFF	midi	Microsoft format midi file

FOREWORD

SAMPLE

A sample is a recorded sound with the maximum harmonic amount and the maximum flat amplitude curve (no decay).

Samples can be *looped* (piano, flute, guitar...helicopter) or *one shot* (drum, conga, cymbals... car crash)

When playing a sample with a keyboard, a *looped* sample sounds as long as the key is on, a *one shot* sample sounds during a fixed time independent of the key pressed time.

INSTRUMENT

The MIDI standard defines for a note 3 main parameters:

the note number (frequency)

the velocity (volume and filter amount)

the note on, note off message (duration)

An instrument is a set of samples playing through a variable filter and amplifier.

A sample can be assigned to a range of MIDI note and/or a range of velocity.

The Instrument parameters will affect the amplitude and the filter in real time.

SOUND BANK

The MIDI standard defines also the program change and the bank select messages.

These features allow to select an *instrument* (piano, strings, drums ...) before playing notes.

The MIDI mapping of the *instruments* is called the *sound list*.

The *sound bank* includes the *sound list* the *instruments* and the *samples*

SOUND BANK CONCEPT

The *sound bank* is a set of instruments mapped according to MIDI specification.

SBE can manage up to 8 sound banks.

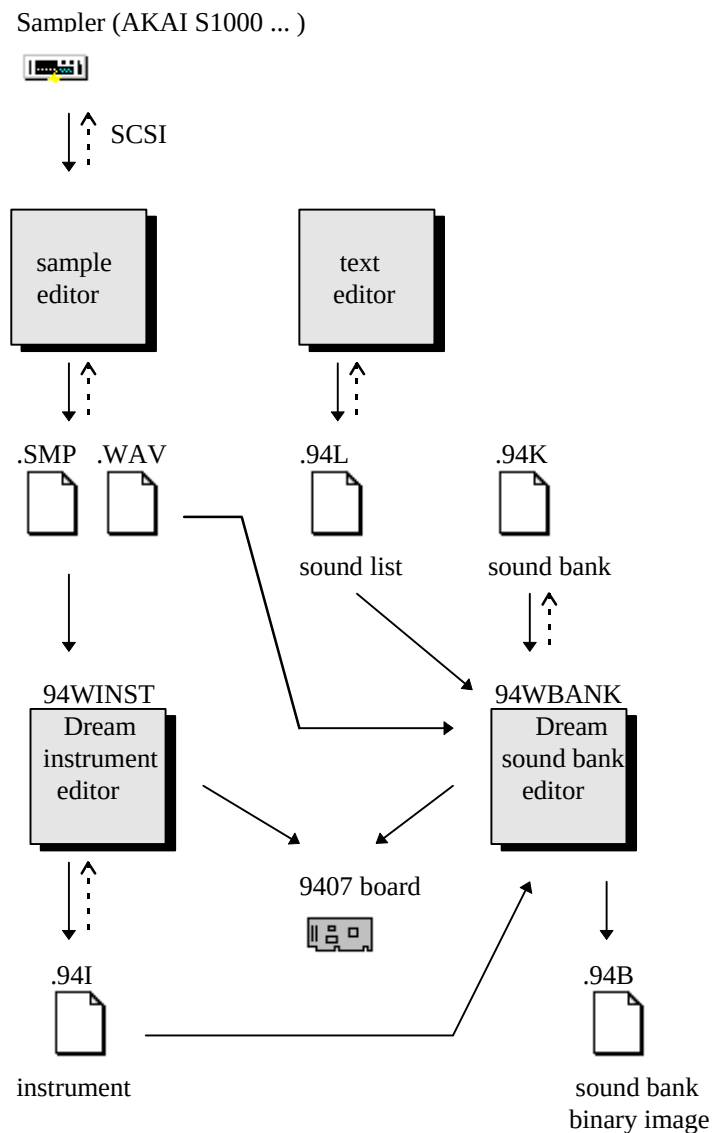
The basic function are insert a sound bank or delete a sound bank.

Because each sound has its own MIDI mapping, mapping conflicts will happen. To avoid this, a sound bank has a priority level (0 to 255) , a new sound bank with highest or equal priority enforces its MIDI mapping.

A typical multimedia application uses a GM factory sound bank plus a user sound bank with additional instruments.

The double sound bank capability allows to create simple user banks without editing and processing factory banks.

9407 SOUND TOOLS SYNOPTIC



NOTE: Dream doesn't provide any sample editor. We recommend to use *SOUND FORGE* from *SONIC FOUNDRY*. Sampling can be done with Sound Forge using 94PnP2 card or using AKAI S1000 with midi dump. Use The 9407smp.exe program (see chapter 1) to convert sample loop pint from *SOUND FORGE* to *SAM9407* format.

".94L" SOUND LIST

The 94L files is an ASCII file created with a text editor.
Sound list describes the MIDI program change and variation mapping .

SYNTAX

Any character following ";" is considered as comment.
Space, tabulation or "," can be used to separate argument.
Command is 4 char long "\$xxx", any unknown command is ignored by software.

HEADER

Sound list begin with header. Header ends with "\$end".
Any header line start with a key word "\$xxx".
\$pro, \$dir & \$bin should always be specified and take place at the beginning of header.
Other parameter are optional (default value in bracket).

\$pro		product name
\$dir		product directory
\$bin	["product"..94B]	Upload file
\$map	["product" .map]	map listing file
\$rep	["product" .rep]	report file
\$lst	["product" .lst]	parameter listing file
\$end		end of header

SOUND LIST FORMAT

After the header, instrument list is described with instrument name lines and command lines.

Command

\$ins *.ins directory, (default dir is current dir), ins directory is used for all *.ins until a new "\$ins".
\$pcm *.16 directory same remarks as \$ins

Instrument

PIANO1 \$ p,[b] p prog nb, b bank nb [default 0]

Drum set

STDDRM1 \$drm p,s,e,[b] p prog nb, s start note, e end note, b bank nb
... (drums list)
\$end end of drum set definition

Drums

STD27 [\$exc x] x exclusion group

".94I" INSTRUMENT DEFINITION

.94I files begin with a HEADER. This header is used by the 94WINST editor.
The header is followed by the INSTRUMENT PARAMETER DEFINITION.

INSTRUMENT HEADER

char ckID[4]="SLIS"	; 4 bytes = "SLIS", sample list information
long ckSize	; 4 bytes = length of sample list
char ckID[4]="SAMP"	; 4 bytes = "SAMP", sample information
long ckSize	; 4 bytes = length of sample information
char Name[13]	; name of sample file in 13 bytes
char Ext[3]	; extension of sample file in 3 bytes
long Smpstart	; 4 bytes = sample start (relative to 1 st sample)
long Smpend	; 4 bytes = sample end (relative to first sample)
int pcm8	; 2 bytes = 08h,00h if 8 bit ; 010h,00h if 16 bit
int SplitID	; 2 bytes = split which is using sample

repeat SAMP block for each different sample. If same sample is used by two different splits, SAMP block must be repeated.

extension recognize by 94WINST are "SMP" and "WAV"

char ckID[4]="KLIS"	; 4 bytes = "KLIS", keyboard tracking list
long ckSize	; 4 bytes = length of keyboard tracking list
char ckID="KBDT"	; 4 bytes = "KBDT", keyboard information
long ckSize	; 4 bytes = length of keyboard information
int SplitID	; 2 bytes = split which is using keyboard
int KbdID	; 2 bytes = keyboard number
char key	; 1 byte= key number
char value	; 1 byte= keyboard value

key/value block number can be from 2 minimum till 4 maximum

repeat KBDT block for each different keyboard tracking table

char ckID[4]="MINS"	; 4 bytes = "MINS", instrument information
long ckSize	; 4 bytes = size of instrument
char data[]	; n bytes, instrument definition (see § 0 PARAMETERS)

PARAMETERS

Instrument definition begins with an header, and then is followed by definition of all splits of the instrument.
All pointers are absolute pointers. All values are 16 bit values (word)

Parameter header

Split definition (3 x n words) :

split 1 start value

split 1 stop value

sound 1 pointer

...

split n start value

split n stop value

sound n pointer

Split 1 start format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
mn								rep	dyn (0-127)						

mn : reserved for future use, must be always 0

rep : repeat bit. If repeat=0, when playing same note number new note kills the old one. If repeat=1, new note is added to old one (typically rep=0 for organ sounds type, rep=1 for piano or timpani sounds type).

Split n start format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0								0	dyn (0-127)						

Split stop format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1								0	dyn (0-127)						

if split start < |0| current note |0| current dyn | <= split stop, then sound pointed by pointer following split value will be played.

Sound pointer format

Absolute pointer to status word (see§)

Note :

This split definition format allows to do layering.

For example, we can define :

key	0...60	61.. 72	73...127
	piano11		piano12
	silence	piano13	

keys from 0-60 will play piano11

keys from 61 till 72 will play piano11+piano13

keys from 73 till 127 will play piano12+piano13

Split definition is :

DW 0000h ; start note = 0, start dyn = 0
DW 0487Fh ; stop note = 72, stop dyn = 127
DW piano11 ; pointer to piano11 sound status word
DW 04800h ; start note = 72, start dyn = 0
DW 07F7Fh ; stop note = 127, stop dyn = 127
DW piano12
DW 03C00h ; start note = 60, start dyn = 0
DW 0FF7Fh ; E =1, stop note= 127, stop dyn = 127
DW piano13

This split definition format allows also to do dynamic split.

For example :

key	Key 0-127
dyn<=64	bass11
dyn>64	bass12

Keys with dynamic <=64 will play bass11 sound, >64 will play bass12 sound.

Split definition is :

DW 0000h ; start note = 0, start dyn = 0
DW 07F40h ; stop note = 127, stop dyn = 64
DW bass11
DW 0040h ; start note = 0, start dyn = 64
DW FF7Fh ; E = 1, stop note = 127, stop dyn = 127
DW bass12

Split format

Ptrtab pointer (1 word)
 Status (1 word)
 Kbd pointers (0 to 4 words) (Kbd = keyboard tracking)
 Eg pointers (1 to 4 words) (Eg = Envelope Generator)
 Mod pointers (0 to 2 words) (Mod = modulator)
 Objects number (1 word)
 Ma1 objects definition
 Ma2 objects definition
 Mb objects definition
 Mx objects definition
 My objects definition
 Modulator definitions
 Enveloppe generator definitions
 Keyboard table definitions
 Ptrtab table definitions

Ptrtab pointer

Pointer to Ptrtab. (see §)

Warning !! : Sound pointer defined in split definition is not pointer to this word but to next word following (Status).

Status

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	2x	alg					0	Eg nb		Mod nb		Kbd nb		

2x : double slot flag (always 0 for time being)

alg : algorithm number (0-31) (see also alglist.wri file)

0 : 24db filter, forward loop

1 : 24db filter, alternate loop

2 : 24db filter, alternate loop with sign inversion

kbdnb : number of keyboard table used : 100=0, 011=1, 010=2, 001=3, 000=4

modnb : number of modulator used : 10=0, 01=1, 00=2

egnb : number of envelope used : 11=1, 10=2, 01=3, 00=4

Algorithm list can change depending on version of program. On first version of program for sound compatibility with 9233, algorithm 0 is 12db filter forward loop instead of 24db filter forward loop. Algorithms 1 and 2 are 24db filters.

Objects number

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MA nb		MA2 nb			MB nb			MX nb			MY nb			Freq off	

MA1 nb : 11=1, 10=2, 01=3, 00=4

MA2 nb : 111=1, 110=2, ... 000=8

MB, MX and MY : 111=0, 110=1, ... , 000=7

Freq Off : Number of 1st frequency object number among MA2 objects

MA1 object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
kbd	0	0	dyn	Filt		Kbd nb		rfu	Lfo	EG	lnb	Egnb		out	off
0	Amplitude (0,127)							dyn offset				dyn slope			
0	Offset (0,127)							lfo amount (-127,+127)							

Kbd : Keyboard table tracking yes/no

Dyn : Dynamic yes/no

Filt : 0 normal, 1 filter input, 2 filter Fc, 3 filter Q

Kbd nb : Keyboard table number (0-3)

Lfo : Modulator yes/no

Eg : Envelope yes/no

Lnb : Modulator number (0-1)

Egnb : Envelope number

Out : Amplitude being general volume of sound

Off : amplitude with offset

Dyn offset, Dyn slope : define curve applied to MIDI dynamic before applying it to amplitude :

Case of slope positive :

slope = 0000,0001,0010,0011,0100,0101,0110,0111

is respectively 0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75

offset = 1000, 1001, 1010, ... 1111, 0000, 0001, 0010, ... 0110, 0111

is respectively -128, -112, -96, ..., -16, 0, +16, +32, ... , +96, +112

Case of slope negative

slope = 1111, 1110, ... 1000

is -0.25, -0.5, ... -2

offset = 1000, 1001, 1010, ... 1111, 0000, 0001, 0010, ... 0110, 0111

is respectively 0, +16, +32, ..., +112, +128, +144, ... , +224, +240

Notes : Word Offset/Lfo amount is present only if either Lfo or Off bits are set

MA2 Object

MA2 frequency object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Lfo amount (-127,+127)								pg	kbd typ		Eg	fmt	Lnb	Eg nb		
detone (-63,+63)								detune (0,127)							typ	
page										Eg amount (-31,+31)						

Typ = 0 to indicate MA2 frequency object

kbd typ : 0 normal, 1 not used, 2 Fix frequency, 3 Fix frequency modulated by kbd0

Eg : Eg yes/no

Lnb : Mod number (0-1)

Eg nb : Eg number (0-3)

pg : 1 if holds bank information

fmt : used for fm sounds. If 1, +64 is added to detone.

NOTES : In case of fix frequency, fix frequency is given in word detone/detune with format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
a9	a8	a7	a6	a5	a4	a3	a2	a1	a0	f5	f4	f3	f2	f1	f0

(40h=nominal frequency)

detone is signed, detune is always positive

word Page/EG amount is present only if Eg or pg bit is set

MA2 Xfer object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR 27-20								PAR 19-12							
PAR 11-0												0	0	0	typ

Typ=1 to indicate MA2 Xfer object type

PAR27-0 are 28 bit value written to SAM.

MB object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR 11-0												0	dum	0	mix
PAR 27-20								PAR19-12							

mix : set if PAR 11-0 holds mix value (effect mix)

dum : set if dummy object (in that case 2nd word is skipped)

MX object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR15-0															

Notes :

If PAR15-0 = 0000h, mx object is not coded, except if one mx object <> 0 is following

PAR15-0 = 0001h means damp object

Damp object must be coded only if one mx object <>0 is following.

MY object

MY Xfer object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR 11-0												0	dum	typ	mix

Typ=0 to indicate MY Xfer type object

Mix : set if parameter holds mix value (main/aux mix)

Dum : set if dummy object

MY Amplitude object

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Amplitude (0,127)							dyn	kbd	0	0	filt		typ	0
dyn offset				dyn slope				0						kbd nb	

Typ=1 to indicate My amplitude type object

Dyn : Dynamic yes/no

Kbd : Keyboard table yes/no

Filt : 0 normal, 1 filter input, 2 filter Fc, 3 filter Q

KBD nb : Keyboard table number

Dyn offset, slope : same as MA1 object

Note : Kbd/dyn word is present only if either dyn or kbd bit is set

Modulator

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Lfo attack (0,127)							0	0	0	0	wave			1
0	Lfo speed (0,127)							0	Lfo delay (0,127)						

Lfo wave : 0 sinus, 1 up saw, 2 down saw, 3 square, 4 random, 5 positive sinus

Envelope generator

Header :

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Init amplitude (0,127)							dyn eg (0-31)				kbd		kbd nb	

Kbd : Keyboard table yes/no

Kbd nb : Keyboard table number

Dyn eg : Dynamic applied on init amplitude, level and rate of each segment

Segment definition :

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
code			Level (0,64)						Rate (0,127)						
Jump back															

Code : 0 continue on next segment

1 loop

2 sustain point, wait for key off

3 envelope end

4 envelope end at sustain point (case of sustain point amplitude = envelope end amplitude)

Jump back : number of segments back in case of loop (if 1 loop on loop segment, if 2 loop one behind loop segment)

Notes : Jump back needed only if code is 1, not present for other codes.

Maximal segment number is 8

Keyboard table

Length of keyboard table depends on split definition. For a sound defined as split from note N till note M (M excluded), keyboard table is (M-N)/2 byte long :

Keyboard table is (M+1)/2 byte long:															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Note N+2, N+3								Note N, N+1							
...															
Note M-2,M-1								Note M-3,M-4							

Note : Keyboard values are from -127 till +127

PTRTAB table

Give indication of position of mix, and modulators data.

word 0 pointer to mix1 value or 0 if none

word 1 mix1 address

word 2 pointer to mix2 value or 0 if none

word 3 mix2 address

word 4 pointer to mix3 value or 0 if none

word 5 mix3 address

word 6 pointer to mix4 value or 0 if none

word 7 mix4 address

word 8 0

word 9 pointer to mod0 pointer or 0

word 10 pointer to mod1 pointer or 0
word 11 0

mix address :

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	block nb			0	0	0	0	0	M/S	ram ad	

block nb = 010 if MB, 100 if MY

ram ad = 0 to 3

M/S = 1 if slave slot in case of double slot algorithm.

Note : all pointers in PTRTAB table are pointers relative to status word in sound definition

".94K" SOUND BANK

The Sound Bank file includes all the source files required by the sound bank: Sound list, Instruments, Samples.

FORMAT

File format is based on RIFF chunk

char	ckID[4]= "PRIO"
long	ckSize
	char name[8]
	word Priority level
char	ckID[4]= "INSX"
long	ckSize
	index table (see §)
char	ckID[4]= "SMPX"
long	ckSize
	index table (see §)
char	ckID[4]= ".94L"
long	ckSize
	"xxxxxxxx.94L" file (see §)
char	ckID[4]= ".94I"
long	ckSize
	"xxxxxxxx.94I" file (see §)
char	ckID[4]= ".SMP"
long	ckSize
	"xxxxxxxx.SMP" file (see §)
char	ckID[4]= ".WAV"
long	ckSize
	"xxxxxxxx.WAV" file (see §)

INSTRUMENT INDEX TABLE

This table allowed direct access to INST chunk

It should take place at the beginning of file just after PRIO chunk

Instrument index definition:

char Name[13]	13 bytes	name of file
char Ext[3]	3 bytes	extension of file
long pt	4 bytes(LSB first)	abs byte pointer to chunk in ".94K"
checksum	long	checksum on first 200bytes

SAMPLES INDEX TABLE

This table allowed direct access to .SMP & .WAV chunk

It should take place after INSX chunk

The samples are sorted by decreasing size.

Sample index definition:

char Name[13]	13 bytes	name of file
char Ext[3]	3 bytes	extension of file
long pt	4 bytes(LSB first)	abs byte pointer to chunk in ".94K"
checksum	long	checksum on first non zero 200bytes
start	long	first used sample (start or loop start)
end	long	last sample
occurrence	word	number of split using sample (0 means that
there is no need to keep sample inside sound bank)		
sample length	word	8 or 16 (only 8 & 16 bits supported)

If SBE needs to insert the same samples with different sample length the sample is duplicated with both length and a warning message is displayed.

".94B" BINARY

The Binary File is the result of compilation of the Sound Bank (.94K).
The Instruments and the samples are stored as SAM9407 reloc format.

Format

File format is based on RIFF chunk

char ckID[4]= "PRIO"
long ckSize
char name[8]
word Priority level

char ckID[4]= "IMAP"
long ckSize
variable *Instrument table*
word FFFFH
variable *drumset table*
word FFFFH

char ckID[4]= "PARA"
long ckSize
94 instruments parameter in reloc format (same as ".94I" without instrument header block (see §))

char ckID[4]= "SAMP"
long ckSize
samples (see §)

char ckID[4]= "RLOC"
long ckSize
param reloc table (see §)

Instrument / drumset table

These tables includes instrument/drumset definition data with the following format:

word program number
byte variation number
byte param page number (0/1)
word 16 bit pointer in param page

SAMPLES

Samples data are stored in either 8 or 16 bit signed format.

Sample data should be inpage 256K word.

Each 512KB page should start with 16 "0" word and end with 16 "0" word. This gaps are necessary to avoid address overflow in DSP address unit.

In the following example there are 4 samples to map: pcm_0 & pcm_1 on 16 bits, pcm_2, pcm3 on 8bits

pcm_0	
pcm_1	
pcm_2	pcm_3
gap	

Samples are mapped on 256K word pages numbered for 1 to n.

Page 1 to n-1 are always considered as full, Page n size is almost always inferior to 256K.

RELOC TABLE

POINTER LIST

instrument parameter (see §) includes many pointers referring to param or samples.

The following list gives the reloc parameter with their associated target (param, samples)

name	§	location in param	mem space	content
sound x pointer		param header	param	point to status word §
PTRTAB pointer		sound	param	pointer to Ptrtab §
KBD pointer		sound	param	pointer to KBD object §
EG pointer		sound	param	pointer to EG object §
MOD pointer		sound	param	pointer to MOD object §
MA2 Xfer obj		MA2 object	samples	MA2[0] PG, MA2[1] LOOP
MB Xfer object		MB object	samples	MB[0] PHI, MB[1] END
pointer to mix		PTRTAB table	param	pointer to mix address
pointer to mod		PTRTAB table	param	pointer to MOD pointer

FORMAT

long pointer point the word in param which is reloc type

byte type

byte block Page nb , 1 or 2 for param , 1 to n for Samples

type	
0	param pointer
1	MA2 [0] PG
2	MA2 [1] LOOP
3	MB[1] END

pointer to full 64K page are always correct (no needs to be reloc)

pointer to MA2[1] or MB[1] full 256K page are also correct.

".SMP" SampleVision SAMPLE FORMAT

This is the Sample format created by Sample vision. This Format enable loop sample.
A User comment has been added to support fractional loop not supported by standard .SMP format.
.SMP format is going to be replaced by .WAV format which is the PC standard.

HEADER

Offset	Size	Description
0 0H	18 12H	'SOUND SAMPLE DATA' ASCII file ID
18 12H	4 4H	'2.1 ' ASCII file version
22 16H	60 3CH	user comment : "Loop"/"Alt" " Size=xxx.xxx ... "
82 52H	30 1EH	Sample name (left justified 30 ASCII char)
112 70H	4 4H	Sample size (sample data count in words. store as double word Intel format)
Total	116 74H	

DATA

Offset	Size	Description
116 74H	2*Size	Sample data signed 16bit integer Intel format
2*Size +116	2 2H	reserved
Total	2*Size+2	

LOOP

They are 8 loop definition

Loop block start at : $118 + 2*Size / 76H + 2*Size$

loop block end at : $205 + 2*Size / CDH + 2*Size$

Offset	Size	Description
0	4	Loop n Start (sample count -> word count)
4	4	Loop n End
8	1	Loop n Type (0 = loop off, 1 = forward, 2 = alternate)
9	2	Loop n Count (times to execute loop before jumping to next loop)
Total	11 BH	

MARKER

They are 8 marker definition

marker block start at : $206 + 2 * \text{Size} / \text{CEH} + 2 * \text{Size}$

Marker block end at : $317 + 2 * \text{Size} / 13\text{DH} + 2 * \text{Size}$

Offset	Size	Description
0	10	Loop n Start (sample count -> word count)
10	4	Loop n End
Total	14 EH	

marker 8 "Start " is used as sample start

MISCELLANEOUS

Misc. block start at : $318 + 2 * \text{Size} / 13\text{EH} + 2 * \text{Size}$

Misc. block end at : $330 + 2 * \text{Size} / 14\text{AH} + 2 * \text{Size}$

Offset	Size	Description
0	1	MIDI Unity Playback Note (MIDI note that will play the sample at its original pitch)
1	4	Sample rate in Hz
5	4	SMPTE Offset in subframes
9	4	Cycle size sample count in one cycle of the sample sound. 1 if unknown
Total	13 DH	